

MISION RESCATE DB

Recuperar un backup sin adivinar nada

Guia paso a paso, con nombres de carpetas, archivos, botones y comandos. Pensada para seguirla como una receta: un paso, una comprobacion, y solo entonces el siguiente.

Regla de oro: todos los archivos del mismo backup llevan exactamente el mismo sello de fecha y hora.

```
database_backup_2026-08-02_00-24-35
```

Regla de seguridad: la identidad privada age nunca se sube a GitHub, a la web, a un chat ni al kit.



Abrir la PWA

[stjames-way.github.io/
backup-recovery-pwa/](https://stjames-way.github.io/backup-recovery-pwa/)

Version 1.0 - 2 de agosto de 2026



Entiende las piezas antes de tocar nada

No todos los archivos hacen lo mismo. Esta tabla evita confundir una firma, un hash, una clave publica y una clave privada.

Pieza	Que es	De donde sale	Secreto
manifests/*.json	Inventario firmado del backup: hashes, partes, formato y origen.	Workflow de GitHub despues de crear y cifrar.	No
signatures/*.sig	Firma Ed25519 real del manifest.	OpenBao Transit mediante backup-signer.	No
signatures/*.json	Metadatos secundarios: proveedor, version de clave y fecha.	Workflow de GitHub.	No
encrypted_backups/*.part	Trozos del archivo cifrado .dump.age.	pg_dump -> age -> split.	Datos cifrados
backup-signing-public-key.pem	Clave publica que verifica la firma .sig.	Parte publica de OpenBao Transit.	No
recovery-trust.json	Politica publica: recipient y huellas aprobadas.	Repositorios DB-backup y PWA.	No
age-identity.txt	Identidad privada que descifra el backup.	Custodia local/offline.	SI



Alto: la clave publica verifica la firma, pero no descifra. La identidad privada descifra, pero no sustituye la verificacion de la firma.

1 Prepara el Mac y comprueba la identidad privada

Antes de descargar 150 MB o mas, asegurate de que tienes la identidad correcta y las herramientas necesarias.

PRIVADA

Identidad local

Ruta usada en este proyecto:

```
$HOME/claves/camino-backup-identity.txt
```

Si la guardaste en otro sitio, usa su ruta real. No generes otra identidad.

PUBLICO ESPERADO

Recipient aprobado

```
age18gt5e7d48tfyh4552kc5wyp52lt5rf34ag0sat3t4xsrg0fqq8stagvae
```

```
f59fd599322f109270cfa7fd614e38b8eb7d5ca823c0443f8f0d55651e4b31aa
```

Instala herramientas si faltan

```
brew install age postgresql@17
export PATH="$(brew --prefix postgresql@17)/bin:$PATH"
```

Comprueba la identidad sin mostrar la clave privada

```
IDENTITY="$HOME/claves/camino-backup-identity.txt"
```

```
RECIPIENT="$(age-keygen -y "$IDENTITY")"
printf 'Recipient: %s\n' "$RECIPIENT"
printf '%s' "$RECIPIENT" | shasum -a 256
```

```
Recipient: age18gt5e7d48tfyh4552kc5wyp52lt5rf34ag0sat3t4xsrg0fqq8stagvae
f59fd599322f109270cfa7fd614e38b8eb7d5ca823c0443f8f0d55651e4b31aa -
```

El guion final - significa "entrada estandar". No pertenece al hash.

Si el recipient o el hash no coinciden, no sigas. Una identidad nueva no puede abrir los backups antiguos.



2 Elige un backup por su fecha y hora

El manifest manda. Primero eliges un manifest; despues buscas todos los archivos con el mismo nombre base.

1. Abre **StJames-way/DB-backup** en GitHub.
2. Abre el selector de rama y elige **backups-signed-latest-30**.
3. Entra en la carpeta **manifests**.
4. Elige la fecha y hora que quieres recuperar.

EJEMPLO

Manifest elegido

```
manifests/  
database_backup_2026-08-02_00-24-35  
.json
```

NOMBRE BASE

Copia este sello

```
database_backup_2026-08-02_00-24-35
```

Truco de niño de 10 años: imagina que la fecha y hora son el apellido de una familia. Manifest, firma y partes deben tener el mismo apellido.

Comprobacion visual

Correcto	Incorrecto
<pre>manifest ...00-24-35.json firma ...00-24-35.sig partes ...00-24-35...part</pre>	<pre>manifest ...00-24-35.json firma ...17-28-51.sig</pre>

No mezcles archivos de dos horas distintas, aunque sean del mismo dia.

En la interfaz actual de GitHub, el selector de ramas esta en la vista del arbol de archivos. La descarga individual se hace desde la vista del archivo mediante la opcion de contenido raw.

3 Descarga todos los archivos de la misma familia

Carpeta del repo	Archivo que buscas	Obligatorio
manifests/	database_backup_...00-24-35.json	Si
signatures/	database_backup_...00-24-35.sig	Si
signatures/	database_backup_...00-24-35.json	No, pero recomendado
encrypted_backups/	database_backup_...00-24-35.dump.age.aaa.part ...aab.part ...todas las que existan	Si, todas
main/config/	backup-signing-public-key.pem	La PWA ya la incluye

Carpeta recomendada en tu ordenador

```
backup-descargado/  
├─ encrypted_backups/  
│   ├─ database_backup_2026-08-02_00-24-35.dump.age.aaa.part  
│   └─ database_backup_2026-08-02_00-24-35.dump.age.aab.part  
├─ manifests/  
│   └─ database_backup_2026-08-02_00-24-35.json  
└─ signatures/  
    ├─ database_backup_2026-08-02_00-24-35.sig  
    └─ database_backup_2026-08-02_00-24-35.json
```

Dos JSON con el mismo nombre: el navegador puede renombrar uno como (1).json si los descargas en la misma carpeta. El manifest contiene schema_version; los metadatos contienen provider y algorithm. Conservar las carpetas evita la confusion.

Si falta una sola parte, la recuperacion no puede continuar.

4 Carga el backup en la PWA

La web trabaja localmente en tu navegador. No envía las partes a GitHub Pages ni a otro servidor.

Abre:

<https://stjames-way.github.io/backup-recovery-pwa/>

1. Pulsa **Nueva comprobación**.
2. Usa **Seleccionar carpeta** y elige backup-descargado/.
3. Si tu navegador no admite carpetas, usa la carga manual.

MANUAL

Manifest JSON

Selecciona el JSON de manifests/. Es obligatorio.

MANUAL

Firma .sig

Selecciona el archivo de signatures/. Es obligatorio.

MANUAL

Metadatos JSON

Selecciona el JSON de firma. Es opcional y aporta una comprobación secundaria.

MANUAL

Partes .part

Selecciona todas las partes de ese sello de fecha.

La clave pública no hace falta cargarla: la PWA incluye la PEM aprobada y comprueba su huella DER antes de usarla.

La PWA nunca debe pedirte el contenido de AGE-SECRET-KEY- Si una web lo pide, detente.



5 Lee el resultado sin confundirte

VERDE

La comprobacion indicada ha terminado correctamente.

AMARILLO

Hay una advertencia: por ejemplo, faltan metadatos opcionales o se recomienda terminal.

ROJO

Algo no coincide. No unas, no descifres y no restaures.

La PWA debe comprobar, en este orden

1. El manifest tiene schema y formato admitidos.
2. El hash del recipient coincide con la politica aprobada.
3. La PEM incluida tiene la huella DER aprobada.
4. La firma Ed25519 valida el manifest canonico.
5. Los metadatos opcionales coinciden, si existen.
6. Existen todas las partes.
7. El tamaño y SHA-256 de cada parte coinciden.
8. Al unirlas, el SHA-256 del `.dump.age` coincide.

Puedes continuar cuando firma, partes y SHA final esten en verde. Un amarillo por metadatos ausentes no invalida la firma principal.

Un rojo en firma, tamaño, parte o hash significa: parar y volver a descargar los archivos correctos.

El resultado `.dump.age` sigue cifrado aunque todo este verde.

6 Descarga el kit completo de terminal

La PWA crea un ZIP pequeño con instrucciones, scripts y evidencia. No duplica las partes grandes.

1. Pulsa **Verificar y generar kit completo de terminal**.
2. Guarda el ZIP en Downloads.
3. Descomprimelo.

```
terminal-kit/  
├─ README.txt  
├─ recovery-plan.json  
├─ recovery-trust.json  
├─ check-age-identity.sh  
├─ verify-and-join.sh  
├─ decrypt-and-check.sh  
├─ check-age-identity.ps1  
├─ verify-and-join.ps1  
├─ decrypt-and-check.ps1  
└─ evidence/  
    ├─ manifest.json  
    ├─ firma.sig  
    ├─ metadatos-opcionales.json  
    ├─ backup-signing-public-key.pem  
    └─ recovery-trust.json
```

Archivo	Para que sirve
README.txt	Receta exacta para ese backup.
recovery-plan.json	Partes, tamaños, orden y hashes de ese backup.
recovery-trust.json	Recipient y huellas publicas aprobadas.
check-age-identity.sh	Comprueba la identidad privada sin mostrarla.
verify-and-join.sh	Verifica y une las partes en un <code>.dump.age</code> .
decrypt-and-check.sh	Descifra, comprueba el dump y ejecuta <code>pg_restore --list</code> .

7 Haz que el kit confirme tu identidad age

Entra en la carpeta descomprimida del kit y da permiso a los scripts:

```
cd "$HOME/Downloads/NOMBRE-DEL-TERMINAL-KIT"

chmod 700 \
  check-age-identity.sh \
  verify-and-join.sh \
  decrypt-and-check.sh
```

Ahora comprueba la identidad:

```
./check-age-identity.sh \
  "$HOME/claves/camino-backup-identity.txt"
```

```
VERDE: identidad age correcta
Recipient: age18gt5e7d48tfyh4552kc5wyp52lt5rf34ag0sat3t4xsrg0fqg8stagvae
SHA-256: f59fd599322f109270cfa7fd614e38b8eb7d5ca823c0443f8f0d55651e4b31aa
```

Que demuestra: la identidad privada local deriva exactamente el recipient publico usado para cifrar estos backups.

Que no demuestra: todavia no ha descifrado el archivo ni restaurado PostgreSQL.

Si aparece "Recipient incorrecto", no fuerces nada y no cambies el manifest. Busca la identidad original correcta.

8 Verifica otra vez y une las partes

Desde la carpeta del kit:

```
./verify-and-join.sh \  
"$HOME/Downloads"
```

El script busca las partes de tres maneras:

1. Downloads/encrypted_backups/archivo.part
2. Downloads/archivo.part
3. Una unica coincidencia dentro de subcarpetas.

```
VERDE: parte 1/2 correcta: encrypted_backups/...aaa.part  
VERDE: parte 2/2 correcta: encrypted_backups/...aab.part  
  
VERDE: BACKUP CIFRADO UNIDO Y VERIFICADO  
Resultado cifrado: ...database_backup_....dump.age  
SHA-256: ...
```

Muy importante: .dump.age significa que el archivo sigue cifrado. Aun no es el dump PostgreSQL utilizable.

Si dice que ya existe el resultado, no lo sobrescribas a ciegas. Borra solo el resultado anterior que tu mismo generaste o indica otro nombre de salida.

9 Descifra y comprueba el dump PostgreSQL

```
./decrypt-and-check.sh \  
"./database_backup_YYYY-MM-DD_HH-MM-SS.dump.age" \  
"$HOME/claves/camino-backup-identity.txt"
```

Sustituye el nombre por el que genero el paso anterior. El script hace:

1. Comprueba otra vez recipient y SHA-256 de la identidad.
2. Descifra con age hacia un archivo temporal.
3. Comprueba el SHA-256 del dump descifrado.
4. Renombra el temporal solo cuando todo coincide.
5. Ejecuta `pg_restore --list` si esta instalado.

```
VERDE: identidad age correcta  
VERDE: dump descifrado y SHA-256 correcto  
VERDE: pg_restore --list correcto  
  
Dump: database_backup_....dump  
Catalogo: database_backup_....dump.contents.txt
```

Ahora tienes un dump PostgreSQL autentico, integro y legible por `pg_restore`.

Todavia falta restaurarlo en una base aislada. Un catalogo valido no prueba que todas las tablas y datos se restauren correctamente.

10

Restaura en PostgreSQL de prueba, nunca en produccion

Este ejemplo usa Docker y PostgreSQL 17 en el puerto local 55432. La contraseña es solo para el contenedor temporal.

1. Levanta una base vacia

```
docker run --name backup-restore-test \  
-e POSTGRES_PASSWORD='local-only-change-me' \  
-e POSTGRES_DB='restore_test' \  
-p 55432:5432 \  
-d postgres:17
```

2. Espera a que PostgreSQL este listo

```
until docker exec backup-restore-test \  
pg_isready -U postgres -d restore_test; do sleep 2; done
```

3. Restaura el dump

```
export PGPASSWORD='local-only-change-me'  
  
pg_restore \  
--verbose \  
--clean \  
--if-exists \  
--no-owner \  
--no-privileges \  
--host 127.0.0.1 \  
--port 55432 \  
--username postgres \  
--dbname restore_test \  
"./database_backup_YYYY-MM-DD_HH-MM-SS.dump"
```

Una base Supabase puede usar extensiones o servicios que no existen en PostgreSQL puro. Este paso demuestra la restauracion logica basica; una prueba Supabase local es mas fiel.

11 Comprueba que hay esquemas, tablas y filas

```
export PGPASSWORD='local-only-change-me'
```

```
psql \  
--host 127.0.0.1 \  
--port 55432 \  
--username postgres \  
--dbname restore_test \  
--command='\dn'
```

```
psql \  
--host 127.0.0.1 \  
--port 55432 \  
--username postgres \  
--dbname restore_test \  
--command='\dt *.*'
```

```
psql \  
--host 127.0.0.1 \  
--port 55432 \  
--username postgres \  
--dbname restore_test \  
--command="SELECT count(*) AS user_tables  
            FROM information_schema.tables  
            WHERE table_schema NOT IN ('pg_catalog','information_schema');"
```

Comprueba tambien datos que conozcas

-  Existe el esquema principal de la aplicacion.
-  Existen tablas criticas.
-  Las tablas no estan vacias cuando no deberian estarlo.
-  Las fechas de los datos tienen sentido.
-  No hubo errores graves en pg_restore.

Cuando termines, elimina el entorno temporal

```
docker rm -f backup-restore-test  
unset PGPASSWORD
```

Solo ahora puedes documentar: "backup restaurado correctamente en un entorno aislado".



Errores frecuentes y que significan

Error	Significado	Que hacer
Falta encrypted_backups/... part	La parte esta en otra carpeta o falta.	Usa el script nuevo, la raiz correcta o vuelve a descargar.
SHA-256 incorrecto	Archivo incompleto, corrupto o de otro backup.	Detente y descarga de nuevo el archivo con el mismo sello.
Firma Ed25519 invalida	Manifest, firma o clave publica no corresponden.	No descifres. Revisa fecha, rama y origen.
Recipient incorrecto	La identidad privada no es la original.	Busca la identidad correcta. No generes otra.
Metadatos JSON ausentes	Falta evidencia secundaria.	Puede continuar en amarillo si la firma real es valida.
pg_restore no esta instalado	Se descifro, pero no se valido el catalogo.	brew install postgresql@17 y repite.
La PWA muestra datos antiguos	Estado anterior o cache del Service Worker.	Pulsa Nueva comprobacion y recarga con Command + Shift + R.

Nunca "arregles" un hash editando el manifest. El manifest esta firmado; cambiarlo rompe la firma.

La mision solo termina con todas las casillas

Marca cada punto en orden. Si una casilla falla, vuelve al paso que la genera.

- ☒ He elegido un manifest por fecha y hora.
- ☒ Manifest, firma y partes tienen el mismo nombre base.
- ☒ La PWA ha validado schema y recipient.
- ☒ La huella de la clave publica es correcta.
- ☒ La firma Ed25519 es valida.
- ☒ Todas las partes tienen tamaño y SHA correctos.
- ☒ El SHA del archivo `.dump.age` es correcto.
- ☒ La identidad privada deriva el recipient aprobado.
- ☒ El backup se ha descifrado y el SHA del dump coincide.
- ☒ `pg_restore --list` reconoce el dump.
- ☒ El dump se ha restaurado en PostgreSQL aislado.
- ☒ He comprobado esquemas, tablas y datos importantes.

Resultado final:

BACKUP CREADO - CIFRADO - FIRMADO - VERIFICADO - DESCIFRADO - RESTAURADO

PWA:

<https://stjames-way.github.io/backup-recovery-pwa/>

Repositorio:

<https://github.com/StJames-way/DB-backup>

Referencias de interfaz GitHub: documentacion oficial sobre selector de ramas y descarga de contenido raw.



Empieza aqui